Advanced Data Computing (405 @ LU) Notes

Abhishek Chakraborty

August 20, 2026

Table of contents

Welcome

These notes have been developed for the CMSC/STAT 405: Advanced Data Computing course at Lawrence University, Appleton, Wisconsin. This course is intended for upper-level undergraduate students who have previously taken a course on R-programming (that covers the `tidyverse` ecosystem, webscraping, and Shiny apps).

We use R, and access it through Posit Cloud, as our primary computing environment, except for writing SQL code. These notes have been heavily adopted and adapted from the following excellent open-source resources:

- R for Data Science (2e) by Hadley Wickham, Mine Çetinkaya-Rundel, and Garrett Grolemund.
- Advanced R by Hadley Wickham .
- Modern Data Science with R (3e) by Benjamin Baumer, Daniel Kaplan, and Nicholas Horton.
- Text Mining with R by Julia Silge and David Robinson.
- R Packages (2e) by Hadley Wickham and Jennifer Bryan.

Besides these wonderful texts, some additional resources are mentioned at the beginning of each module and/or chapter. Some of the prose have been written in collaboration with AI. Reader comments and feedback are greatly appreciated. To report errors or bugs, please post an issue or raise a pull request here.

Part I

SQL

Introduction: Beyond the Flat File

In most introductory statistical and data science coursework (such as CMSC/STAT 205: Data-Scientific Programming), we typically work with *static* data, that is, a (or, a few) `.csv` or `.xlsx` file(s), which we can load entirely into our computer's active memory (RAM) - this is similar to loading dataset(s) into the `Environment` pane in the top right of our Posit Cloud screen.

While this setup is highly effective for isolated research questions and pedagogical exercises inside the classroom, data in the real-world, or simply, datasets, are rarely static, and they are almost always larger than what can fit into a single machine's memory. This is common in most industry environments and complex organizational structures.

This is where **Database Management Systems (DBMS)** become necessary. A DBMS is a software system designed to store data in a structured format, and which can also be used to retrieve, and manage data efficiently and securely. Moving from flat (static) files to a robust database framework solves three fundamental infrastructural problems:

1. **Scale:** Databases are optimized to query gigabytes or terabytes of data dynamically without requiring the entire dataset to be loaded into our local environment.
2. **Concurrency:** A flat file cannot easily be read and written to by hundreds of users simultaneously without data corruption. Databases handle concurrent actions safely, that is, allow for multiple users to access the data simultaneously, maintaining a single source of truth.
3. **Integrity:** Databases enforce strict rules about what data can enter the system (schemas), ensuring that a numeric column cannot accidentally ingest a string of text.

In a database, data are stored in the form of tables. Conceptually, a table in a database is no different from the data frames which are used in `R`. They have the same rectangular format with horizontal rows and vertical columns. However, there are two major differences between a data frame that we are used to (from 205) and a table in a database.

1. **Size**: Tables in a database are too large to be stored in our computer's memory unlike typical data frames we used previously. Thus, we need to store tables remotely on a hard drive outside of our personal computers. This, in turn, make the data secure and can be accessed by multiple users at a time.
2. **Relationality**: Tables in a database are usually linked with a **key**.

The Relational Revolution

Prior to the 1970s, data storage was highly rigid. Systems used hierarchical or network models, meaning that to extract data, a programmer had to write complex procedural code detailing

the exact physical navigation path through the computer's storage layer. If the physical storage hardware changed, the software broke.

In 1970, Edgar F. Codd, an IBM computer scientist, published a landmark paper: *A Relational Model of Data for Large Shared Data Banks*. Codd proposed separating the physical storage of data from its logical organization. Data would be organized into relations (tables) consisting of tuples (rows) and attributes (columns). Connections between data were established through shared keys rather than physical pointers.

To interact with this new relational model, IBM developed `SEQUEL` (Structured English Query Language) in the mid-1970s, which was later shortened to `SQL` (Structured Query Language). `SQL` became an ANSI standard in 1986 and fundamentally changed how humans interact with data systems.

To digest the concept of relational databases, let's think of this setup. Suppose Lawrence has been storing the records of each student from every graduating class. So, unsurprisingly, that's a **lot** of data. Now, think about how do we want to store these data - say we store students names, their IDs, date of births, classes taken every term, etc. Do we want to store all these data into a single file? Definitely not, for a variety of reasons. Firstly, we would want to separate certain pieces of information. Data about names, date of births, IDs, for example, are unlikely to change. We would like to keep them in one table. Data about courses can be kept in a separate table. These two tables would be related by a key. Now, do we want the key to be student names or their IDs? Which one is more efficient to store in terms of individual entries? Secondly, storing data in the above way lets us update only one table (containing information about courses) every term without touching the other table (student demographics which are unlikely to change). That is more efficient and secure to manage as well. Thirdly, what happens if a student does change their name? Then we can update only one entry in one table, rather than updating multiple entries (if demographic data and course-specific data were stored in a combined table).

`SQL` as a Declarative Language

Most general-purpose programming languages we encounter are *imperative*. We write sequential instructions telling the computer exactly *how* to achieve a result.

`SQL` is fundamentally different; it is a *declarative* language. When we write `SQL`, we describe *what* result we want, and the database engine's query optimizer decides the most efficient algorithmic path to retrieve it. We do not write loops or dictate the order of operations. We state our logical requirements, and the engine handles the execution plan.

This abstraction makes `SQL` incredibly powerful. A query written to extract ten rows from a table of a thousand records uses the exact same syntax as a query operating on a table of ten billion records distributed across a cloud cluster.

SQL is a query language meant for working with relational databases. There exists many dialects of SQL (just like a regular spoken language). Translating between the dialects is not always easy although once we learn how to program in one dialect, we will be able to (hopefully) pick up any of the other SQL dialects. We will use the MySQL dialect in the upcoming chapters, unless mentioned otherwise. MySQL is among the most popular implementations of SQL and it is open source. MySQL is based on a **client-server model**, meaning, the data live on a powerful computer (the server) and we connect to the data from our own computer (the client).

SQL vs. R

R is a statistical computing environment that is developed for the purpose of data analysis. SQL is designed as a robust solution for data management and extraction, its analytic capabilities are very limited. Although, we will notice that there are similarities in the ways we think about and write R and SQL codes, they serve fundamentally different purposes. In the next chapters, we will alternate between SQL and R code, and use them for their specific tasks (for example, we will extract data from a database using SQL and visualize them using R). It's important for a data scientist to have both languages in their toolkit.

Industry Ubiquity

Despite being decades old, SQL is the undisputed *lingua franca* of modern data work. While specialized frameworks rise and fall, SQL remains the underlying connective tissue for data engineering, analytics, and applied statistics.

In contemporary industry contexts, SQL is used to:

- Extract subsets of data from enterprise data warehouses (like Snowflake, Amazon Redshift, or Google BigQuery) before bringing the manageable subset into an analytical environment for modeling.
- Define metrics and dashboards in business intelligence tools.
- Perform robust, in-database feature engineering prior to training predictive models.

Mastering SQL is not just about learning a syntax; it is about learning how to think in sets and relations. It bridges the gap between ad-hoc statistical analysis and production-grade data infrastructure.

Cautionary Note

In the next four chapters, we will use both `R` and `SQL` code chunks. Shown below are an example of each; notice (1) the header which defines the language (although this would be invisible later on), and (2) the difference in how to comment out code, that should be your cue to identify the specific code chunk.

R Code Chunk

````
```{r}

# your R code below

```
````

SQL Code Chunk

````
```{sql}
#| connection: connection_to_your_server

-- your SQL code below

```
````

Further Resources

Besides the resources mentioned at the beginning of the text, I suggest you check out

- Data Science, the SQL notes by Jo Hardin.
- Notes from Database Systems at UC Berkeley
- Introduction to dbplyr
- Hands-on Database (2e) - Steve Conger
- SQL in a Nutshell by Kline et al.
- Documentations for different `SQL` dialects - MySQL, PostgreSQL, and SQLite

1 Accessing Databases and Database Tables

1.1 load packages

To interact with a database using R, we need a specific set of tools. The tidyverse collection of R packages provides our standard data manipulation framework, while DBI (Database Interface) serves as the common architecture for communicating with **Relational DataBase Management Systems (RDBMS)**. Because different databases (PostgreSQL, MySQL, SQLite) speak slightly different dialects of SQL, we also need a specific driver. In this case, we use RMariaDB to communicate with MySQL/MariaDB servers. Finally, dbplyr is the critical translation layer that allows us to write dplyr code that R automatically converts into SQL.

```
library(tidyverse)
library(mdsr)
library(DBI)
library(RMariaDB)
library(dbplyr)
```

1.2 databases and accessing them

Relational databases operate on a **client-server** model. In this architecture, the data does not live on our local machine. Instead, it is housed on a centralized remote server. Our R session (the client) must send a formal request over a network to access that data.

We initialize this connection using the mdsr package, which provides a convenient wrapper function to connect to a public, read-only educational database.

Here, con_air is the connection to the airlines database on the scidb server located on Amazon Web Services. Check out more details on the help page (?mdsr::dbConnect_scidb).

```
# connecting to the airlines database

con_air <- mdsr::dbConnect_scidb("airlines")

# remember the name of the connection
```

> **The Client-Server Analogy**
>
> Think of the **client-server** model like a restaurant. We (the client) sit at a table and look at the menu. We do not go into the kitchen to cook the food ourself. Instead, we send our order (the query) via a waiter (the connection) to the kitchen (the server). The kitchen processes the heavy lifting and sends back only our specific meal (the results).

While wrapper functions like the one above are convenient for teaching, they hide the underlying mechanics. In an industry setting, connecting to a database requires explicit routing and authentication parameters. We must specify the exact server location, the name of the database, our username, and our password.

Here is what an explicit connection via `DBI` looks like:

```
# access wai database

con_wai <- DBI::dbConnect(
  RMariaDB::MariaDB(),
  dbname = "wai",
  host = "scidb.smith.edu",
  user = "waiuser",
  password = "smith_waiDB")
```

> **Password Security**
>
> It is not ideal and hence, not recommended, to share passwords. The code chunk above is shown only for teaching purposes.

1.3 database tables and accessing them

Once a connection is established, the first step is typically reconnaissance: we need to know what tables exist within the database.

We can achieve this using `R` syntax via the `DBI` package:

```
# lists tables in the airlines database

DBI::dbListTables(con_air)
```

```
[1] "airports"        "planes"          "carriers"        "flights_summary"
[5] "flights"
```

Alternatively, because Quarto is a polyglot authoring framework, we can write raw `SQL` directly. By specifying `connection = "con_air"` or using `#| connection: con_air` in the chunk options, Quarto knows to pass the code inside the chunk directly to the database rather than to the `R` interpreter.

Note, the following is a `SQL` code chunk.

```
-- lists tables in the database

SHOW TABLES;
```

Table 1.1: 5 records

Tables_in_airlines
airports
carriers
flights
flights_summary
planes

> **SQL Syntax Conventions**
>
> Notice that we wrote SHOW TABLES in all capital letters and ended the statement with a semicolon (;). While `SQL` is technically case-insensitive, capitalizing `SQL` keywords is a universal standard that drastically improves readability, distinguishing commands from table and column names. The semicolon acts as a terminator, signaling to the server that the query is complete and ready for execution.

Knowing the tables exist is not enough; we must establish a reference to a specific table to query it. We do this using `dplyr::tbl()`. This creates a lazy reference to a remote database table without downloading it. Operations on this object are translated to `SQL` and executed on the server only when results are actually needed.

Here, we query the `flights` table from the established connection to the `airlines` database.

```
# reference to flights table from the airlines database

flights <- dplyr::tbl(con_air, "flights")
```

> **Object Distinctions**
>
> It's easy to conflate the objects we have created thus far. Notice the distinction between `con_air` (the connection object), `airlines` (the database), and `flights` (the table reference). The following hierarchy would be helpful
>
> server → database → table → table reference

Notice the output of `class(flights)`. It is not a standard `data.frame` or `tbl_df`. It is a `tbl_dbi` and `tbl_sql`. This distinction is profound because it introduces the concept of lazy evaluation.

```
# inspect the object type

class(flights)
```

```
[1] "tbl_MariaDBConnection" "tbl_dbi"               "tbl_sql"
[4] "tbl_lazy"              "tbl"
```

When we print the `flights` object below, it provides a quick glimpse of the columns and the first few rows. However, we will notice it says `??` for the total number of rows. This is because the data is not actually in R's memory. `R` has merely established a pointer to the table on the server. Because counting tens of millions of rows takes computational time, the database engine skips that step until explicitly commanded otherwise. It is **"lazy"**—it does as little work as possible.

```
# attempting to print the flights object

flights
```

```
# A query:  ?? x 21
# Database: mysql  [mdsr_public@mdsr.crcbo51tmesf.us-east-2.rds.amazonaws.com:3306/airlines]
    year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
   <int> <int> <int>    <int>          <int>     <int>    <int>          <int>
 1  2013    10     1        2             10        -8      453            505
 2  2013    10     1        4           2359         5      730            729
 3  2013    10     1       11             15        -4      528            530
 4  2013    10     1       14           2355        19      544            540
 5  2013    10     1       16             17        -1      515            525
 6  2013    10     1       22             20         2      552            554
 7  2013    10     1       29             35        -6      808            816
```

```
 8  2013    10     1       29             35        -6      449            458
 9  2013    10     1       31             30         1      519            538
10  2013    10     1       32             33        -1      557            606
# i more rows
# i 13 more variables: arr_delay <int>, carrier <chr>, tailnum <chr>,
#   flight <int>, origin <chr>, dest <chr>, air_time <int>, distance <int>,
#   cancelled <int>, diverted <int>, hour <int>, minute <int>, time_hour <dttm>
```

Lazy tbl

Notice that typical exploratory functions such as `str` and `glimpse` would work differently with the `flights` object compared to datasets loaded in the local session memory.

```
str(flights)
```

```
List of 3
 $ con       :Formal class 'MariaDBConnection' [package "RMariaDB"] with 7 slots
  .. ..@ ptr                   :<pointer: 0x625ded856aa0>
  .. ..@ host                  : chr "mdsr.crcbo51tmesf.us-east-
2.rds.amazonaws.com"
  .. ..@ db                    : chr "airlines"
  .. ..@ load_data_local_infile: logi FALSE
  .. ..@ bigint                : chr "integer64"
  .. ..@ timezone              : chr "UTC"
  .. ..@ timezone_out          : chr "UTC"
 $ src       :List of 2
  ..$ con  :Formal class 'MariaDBConnection' [package "RMariaDB"] with 7 slots
  .. .. ..@ ptr                   :<pointer: 0x625ded856aa0>
  .. .. ..@ host                  : chr "mdsr.crcbo51tmesf.us-east-
2.rds.amazonaws.com"
  .. .. ..@ db                    : chr "airlines"
  .. .. ..@ load_data_local_infile: logi FALSE
  .. .. ..@ bigint                : chr "integer64"
  .. .. ..@ timezone              : chr "UTC"
  .. .. ..@ timezone_out          : chr "UTC"
  ..$ disco: NULL
  ..- attr(*, "class")= chr [1:4] "src_MariaDBConnection" "src_dbi" "src_sql" "src"
 $ lazy_query:List of 5
  ..$ x         : 'dbplyr_table_path' chr "`flights`"
  ..$ vars      : chr [1:21] "year" "month" "day" "dep_time" ...
  ..$ group_vars: chr(0)
  ..$ order_vars: NULL
```

```
  ..$ frame     : NULL
  ..- attr(*, "class")= chr [1:3] "lazy_base_remote_query" "lazy_base_query" "lazy_query"
 - attr(*, "class")= chr [1:5] "tbl_MariaDBConnection" "tbl_dbi" "tbl_sql" "tbl_lazy" ...
```

```
glimpse(flights)  # notice the ?? on the number of rows
```

```
Rows: ??
Columns: 21
$ year           <int> 2013, 2013, 2013, 2013, 2013, 2013, 2013, 2013, 2013, 2~
$ month          <int> 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10,~
$ day            <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
$ dep_time       <int> 2, 4, 11, 14, 16, 22, 29, 29, 31, 32, 33, 34, 38, 39, 4~
$ sched_dep_time <int> 10, 2359, 15, 2355, 17, 20, 35, 35, 30, 33, 35, 2328, 4~
$ dep_delay      <int> -8, 5, -4, 19, -1, 2, -6, -6, 1, -1, -2, 66, -2, -
1, -5~
$ arr_time       <int> 453, 730, 528, 544, 515, 552, 808, 449, 519, 557, 445, ~
$ sched_arr_time <int> 505, 729, 530, 540, 525, 554, 816, 458, 538, 606, 457, ~
$ arr_delay      <int> -12, 1, -2, 4, -10, -2, -8, -9, -19, -9, -
12, 59, -7, -~
$ carrier        <chr> "AA", "FL", "AA", "AA", "UA", "UA", "US", "AS", "DL", "~
$ tailnum        <chr> "N201AA", "N344AT", "N3KMAA", "N3ENAA", "N38473", "N458~
$ flight         <int> 2400, 710, 1052, 2392, 1614, 291, 436, 108, 1896, 687, ~
$ origin         <chr> "LAX", "SFO", "SFO", "SEA", "LAX", "SFO", "LAX", "ANC",~
$ dest           <chr> "DFW", "ATL", "DFW", "ORD", "IAH", "IAH", "CLT", "SEA",~
$ air_time       <int> 149, 247, 182, 191, 157, 188, 256, 181, 147, 183, 177, ~
$ distance       <int> 1235, 2139, 1464, 1721, 1379, 1635, 2125, 1448, 1399, 1~
$ cancelled      <int> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0~
$ diverted       <int> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0~
$ hour           <int> 0, 23, 0, 23, 0, 0, 0, 0, 0, 0, 0, 23, 0, 0, 0, 0, 1, 0~
$ minute         <int> 10, 59, 15, 55, 17, 20, 35, 35, 30, 33, 35, 28, 40, 40,~
$ time_hour      <dttm> 2013-10-01 00:10:00, 2013-10-01 23:59:00, 2013-
10-01 0~
```

To understand the architecture of the table without pulling data, we can use the `DESCRIBE` command in `SQL`. This returns column/field names, data types, and constraints for a table. This is a recommended first step before working with the table.

```
DESCRIBE flights;
```

Table 1.2: Displaying records 1 - 10

Field	Type	Null	Key	Default	Extra
year	smallint(4)	YES		NA	
month	smallint(2)	YES		NA	
day	smallint(2)	YES		NA	
dep_time	smallint(4)	YES		NA	
sched_dep_time	smallint(4)	YES		NA	
dep_delay	smallint(4)	YES		NA	
arr_time	smallint(4)	YES		NA	
sched_arr_time	smallint(4)	YES		NA	
arr_delay	smallint(4)	YES		NA	
carrier	varchar(2)	NO			

This output is conceptually similar to R's `glimpse()` function, but it exposes the underlying database types (e.g., `varchar`, `int`, `datetime`). Databases require strict typing to optimize physical storage on disk. A column defined as `varchar(3)` can only ever hold text strings up to three characters long, preventing data corruption and saving immense amounts of storage space.

Remember, earlier we were trying to figure out if it would be more efficient to store student names versus their IDs!

To fully appreciate the power of lazy evaluation, we can compare the memory footprint of a database pointer versus a materialized in-memory object in `R`. Let us create a reference to the `carriers` table.

```
# reference to carriers table from the airlines database

carriers <- dplyr::tbl(con_air, "carriers")
```

```
DESCRIBE carriers;
```

Table 1.3: 2 records

Field	Type	Null	Key	Default	Extra
carrier	varchar(7)	NO	PRI		
name	varchar(255)	NO			

Let's check how much RAM this connection occupies in our local environment:

```
# carriers lives in the SQL database and is linked remotely

carriers |>
  utils::object.size() |>
  print(units = "Kb")
```

```
7.4 Kb
```

The object size is remarkably small (often just a few kilobytes). It only stores the connection string and metadata, not the data itself.

However, in a typical data science workflow, we eventually need the data in our local environment—usually after we have aggregated or filtered it—so we can pass it to local functions like `ggplot2` for visualization. To bridge the gap between the remote database and our local `R` session, we use the `collect()` function.

```
carriers_df <- carriers |>  dplyr::collect()
```

Let's check the memory size now:

```
# carriers lives in memory in R as carriers_df

carriers_df |>
  utils::object.size() |>
  print(units = "Kb")
```

```
234.8 Kb
```

The materialized data frame is significantly larger because the actual data now resides in our computer's RAM.

> ⚠ The Danger of Collect
>
> We should never run `collect()` on a massive, unaggregated table like `flights`. That table contains hundreds of millions of rows. If we attempt to `collect()` the entire table, we will exhaust our machine's RAM, and our `R` session will crash. The proper workflow is to use `dbplyr` or `SQL` to filter and summarize the data on the remote server, and only `collect()` the finalized, summarized results into R.
> Another safer alternative (if needed) is

```
# flights_df lives in memory in R

flights_df <- flights |>
  head(5) |>
  collect()
```

Just as we would close a file after editing it, we must explicitly sever our connection to a remote database when our analysis is complete. Open connections consume resources on both our local machine and the remote server.

```
# terminate the SQL connection

DBI::dbDisconnect(con_air, shutdown = TRUE)
```

Terminating connections is an essential habit for responsible computing, particularly when sharing server resources in an industry or academic environment.

2 Writing SQL queries

In our previous chapter, we established how to connect R to a remote relational database. However, simply connecting to a database is only the first step. Our primary goal is to extract, filter, and aggregate data to answer analytical questions.

Because we are operating within the R ecosystem but interacting with a SQL database, we have three distinct methods for sending instructions to the server:

1. Translating R's dplyr syntax into SQL using the dbplyr package.
2. Passing raw SQL strings through R functions using the DBI package.
3. Writing native SQL directly within Quarto code chunks.

In this chapter, we will explore all three methodologies, gradually transitioning from the familiar tidyverse syntax to writing robust, native SQL.

2.1 load packages and connect to database server

We begin by loading our required packages and establishing our connection to the airlines database, just as we did previously. We will also create a reference pointer to the flights table, which contains millions of commercial flight records.

```
library(tidyverse)
library(mdsr)
library(DBI)
library(RMariaDB)
library(dbplyr)
```

```
# connect to the airlines database

con_air <- mdsr::dbConnect_scidb("airlines")

DBI::dbListTables(con_air)   # list tables in database
```

```
[1] "airports"        "planes"          "carriers"        "flights_summary"
[5] "flights"
```

```
# reference to flights table from the airlines database

flights <- dplyr::tbl(con_air, "flights")
```

```
DESCRIBE flights;
```

Table 2.1: Displaying records 1 - 10

Field	Type	Null	Key	Default	Extra
year	smallint(4)	YES		NA	
month	smallint(2)	YES		NA	
day	smallint(2)	YES		NA	
dep_time	smallint(4)	YES		NA	
sched_dep_time	smallint(4)	YES		NA	
dep_delay	smallint(4)	YES		NA	
arr_time	smallint(4)	YES		NA	
sched_arr_time	smallint(4)	YES		NA	
arr_delay	smallint(4)	YES		NA	
carrier	varchar(2)	NO			

2.2 SQL in R: Part 1 - the 'dbplyr' R package will directly translate 'dplyr' code into SQL

For data scientists already comfortable with the `tidyverse`, the `dbplyr` package is an extraordinary tool. It acts as a translation engine, taking our standard `dplyr` verbs (`filter`, `arrange`, `select`, `mutate`, `summarize`, `group_by`) and silently converting them into the specific `SQL` dialect required by our database (in this case, `MariaDB/MySQL`).

Suppose we want to find the minimum and maximum year recorded in our `flights` table. We can write standard `dplyr` code:

```
yrs <- flights |>
  summarize(min_year = min(year, na.rm = TRUE),
            max_year = max(year, na.rm = TRUE))

yrs
```

```
# A query:  ?? x 2
# Database: mysql  [mdsr_public@mdsr.crcbo51tmesf.us-east-2.rds.amazonaws.com:3306/airlines]
```

```
  min_year max_year
     <int>    <int>
1     2013     2015
```

The result `yrs` is a **lazy** query — it holds the `SQL` definition, not the data itself.

The data is processed on the remote server, and only the summarized one-row result is returned to our console. But how did `R` communicate this to the database? We can peek under the hood using the `show_query()` function. This reveals the SQL that `dbplyr` generated under the hood.

```
dplyr::show_query(yrs)
```

```
<SQL>
SELECT MIN(`year`) AS `min_year`, MAX(`year`) AS `max_year`
FROM `flights`
```

> **The SQL Vocabulary**
>
> Looking at the output of `show_query()`, we see several foundational `SQL` keywords:
>
> - `SELECT`: Determines which columns or calculations we want to retrieve.
> - `AS`: Creates an alias, similar to the = sign in a `mutate()` or `summarize()` function.
> - `FROM`: Specifies the table we are querying.
>
> Notice also that the query ends with a semicolon (;).

Let us try a more complex example. We want to extract all flights from 2013 that traveled between Los Angeles (LAX) and Boston (BOS).

```
la_bos <- flights |>
  filter(year == 2013 &
           ((origin == "LAX" & dest == "BOS") |
           (origin == "BOS" & dest == "LAX")))

la_bos
```

```
# A query:  ?? x 21
# Database: mysql  [mdsr_public@mdsr.crcbo51tmesf.us-east-2.rds.amazonaws.com:3306/airlines]
    year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
```

```
   <int> <int> <int>    <int>          <int>     <int>    <int>          <int>
 1  2013    10     1      655            700        -5     1508           1530
 2  2013    10     1      704            710        -6     1006           1035
 3  2013    10     1      707            715        -8     1000           1030
 4  2013    10     1      755            800        -5     1611           1626
 5  2013    10     1      800            800         0     1054           1117
 6  2013    10     1      804            800         4     1630           1630
 7  2013    10     1      838            844        -6     1706           1716
 8  2013    10     1     1108           1110        -2     1421           1425
 9  2013    10     1     1202           1205        -3     2015           2040
10  2013    10     1     1208           1209        -1     2016           2035
# i more rows
# i 13 more variables: arr_delay <int>, carrier <chr>, tailnum <chr>,
#   flight <int>, origin <chr>, dest <chr>, air_time <int>, distance <int>,
#   cancelled <int>, diverted <int>, hour <int>, minute <int>, time_hour <dttm>
```

When we inspect the SQL translation for this filter, we notice some critical syntactical differences between R and SQL.

```
dplyr::show_query(la_bos)
```

```
<SQL>
SELECT *
FROM `flights`
WHERE (`year` = 2013.0 AND ((`origin` = 'LAX' AND `dest` = 'BOS') OR (`origin` = 'BOS' AND `
```

In the SQL translation, our R `filter()` statement becomes a SQL `WHERE` clause. Furthermore, notice that SQL uses a single = for logical evaluation, whereas R requires ==. Finally, our symbolic operators & and | are translated into explicit `AND` and `OR` keywords.

Once we are satisfied with our query and wish to bring the data into our local R environment for visualization or modeling, we append the `collect()` function. This should be done only when the result set is small enough to fit in memory.

```
la_bos <- la_bos |>
  collect()

la_bos
```

```
# A tibble: 7,227 x 21
    year month   day dep_time sched_dep_time dep_delay arr_time sched_arr_time
```

```
   <int> <int> <int>    <int>          <int>     <int>    <int>          <int>
 1  2013    10     1      655            700        -5     1508           1530
 2  2013    10     1      704            710        -6     1006           1035
 3  2013    10     1      707            715        -8     1000           1030
 4  2013    10     1      755            800        -5     1611           1626
 5  2013    10     1      800            800         0     1054           1117
 6  2013    10     1      804            800         4     1630           1630
 7  2013    10     1      838            844        -6     1706           1716
 8  2013    10     1     1108           1110        -2     1421           1425
 9  2013    10     1     1202           1205        -3     2015           2040
10  2013    10     1     1208           1209        -1     2016           2035
# i 7,217 more rows
# i 13 more variables: arr_delay <int>, carrier <chr>, tailnum <chr>,
#   flight <int>, origin <chr>, dest <chr>, air_time <int>, distance <int>,
#   cancelled <int>, diverted <int>, hour <int>, minute <int>, time_hour <dttm>
```

⚠ Where `dbplyr` translation breaks down

Notice the code below. Depending on the database backend, this either fails outright or produces a warning, because `cumsum()` requires a `SQL` **window function**, and not every `dplyr` function has a clean, universal `SQL` equivalent — especially once we leave basic aggregation and move into ordered, row-by-row calculations. This is precisely the kind of situation where dropping down to raw `SQL` becomes necessary rather than a stylistic choice. In the sections that follow, we introduce two ways to do exactly that.

```r
# attempting a window function (running total) using dplyr syntax

dbplyr_fail <- flights |>
  filter(origin == "ATW") |>
  group_by(year) |>
  summarize(num_flights = n()) |>
  mutate(running_total = cumsum(num_flights))


dplyr::show_query(dbplyr_fail)
```

```
Warning: Windowed expression `SUM(`num_flights`)` does not have explicit order.
i Please use `arrange()`, `window_order()`, or `.order` to make deterministic.

<SQL>
SELECT *, SUM(`num_flights`) OVER (ROWS UNBOUNDED PRECEDING) AS `running_total`
FROM (
```

```
  SELECT `year`, COUNT(*) AS `num_flights`
  FROM `flights`
  WHERE (`origin` = 'ATW')
  GROUP BY `year`
) AS `q01`
```

2.3 SQL in R: Part 2 - using the DBI R package to write SQL queries in r code chunks

While `dbplyr` is incredibly convenient, it cannot translate every possible `SQL` function, especially highly specialized window functions or dialect-specific commands. In these instances, we can bypass the translation layer and pass raw `SQL` strings directly to the database using `DBI::dbGetQuery()`.

```
# retrieves the first 8 rows from flights

DBI::dbGetQuery(con_air,
                "SELECT * FROM flights LIMIT 8;")
```

```
  year month day dep_time sched_dep_time dep_delay arr_time sched_arr_time
1 2013    10   1        2             10        -8      453            505
2 2013    10   1        4           2359         5      730            729
3 2013    10   1       11             15        -4      528            530
4 2013    10   1       14           2355        19      544            540
5 2013    10   1       16             17        -1      515            525
6 2013    10   1       22             20         2      552            554
7 2013    10   1       29             35        -6      808            816
8 2013    10   1       29             35        -6      449            458
  arr_delay carrier tailnum flight origin dest air_time distance cancelled
1       -12      AA  N201AA   2400    LAX  DFW      149     1235         0
2         1      FL  N344AT    710    SFO  ATL      247     2139         0
3        -2      AA  N3KMAA   1052    SFO  DFW      182     1464         0
4         4      AA  N3ENAA   2392    SEA  ORD      191     1721         0
5       -10      UA  N38473   1614    LAX  IAH      157     1379         0
6        -2      UA  N458UA    291    SFO  IAH      188     1635         0
7        -8      US  N551UW    436    LAX  CLT      256     2125         0
8        -9      AS  N402AS    108    ANC  SEA      181     1448         0
  diverted hour minute           time_hour
1        0    0     10 2013-10-01 00:10:00
```

```
2        0   23     59 2013-10-01 23:59:00
3        0    0     15 2013-10-01 00:15:00
4        0   23     55 2013-10-01 23:55:00
5        0    0     17 2013-10-01 00:17:00
6        0    0     20 2013-10-01 00:20:00
7        0    0     35 2013-10-01 00:35:00
8        0    0     35 2013-10-01 00:35:00
```

In SQL, the * symbol is a wildcard meaning "all columns." The LIMIT clause is the SQL equivalent of R's head() function, restricting the output size.

We can also perform complex aggregations directly within the string. Here, we count the number of flights per year and order the results.

```
DBI::dbGetQuery(con_air,
  "SELECT year, count(*) AS num_flights FROM flights GROUP BY year ORDER BY num_flights;")
```

```
  year num_flights
1 2015     5819079
2 2014     5819811
3 2013     6369482
```

2.4 SQL in R: Part 3 - writing SQL queries in sql code chunks

Passing SQL strings inside R functions can become visually cumbersome for long, multi-line queries. Because Quarto is a polyglot environment, we can change the code chunk type from {r} to {sql}. This provides proper syntax highlighting and allows us to write native SQL exactly as we would in a dedicated database client.

Let us recreate our previous dbplyr queries using native SQL chunks.

```
-- display minimum and maximum year

SELECT MIN(year) AS min_year,
       MAX(year) AS max_year
FROM flights;
```

Table 2.2: 1 records

min_year	max_year
2013	2015

```
-- extract flights between LAX and BOS in 2013

SELECT *
FROM flights
WHERE (year = 2013 AND
      ((origin = 'LAX' AND dest = 'BOS') OR
      (origin = 'BOS' AND dest = 'LAX')));

-- notice the use of = (not ==) for equality comparisons
```

Table 2.3: Displaying records 1 - 10

year	month	day	dep_time	sched_dep_time	dep_delay	arr_time	sched_arr_time	arr_delay	carrier	tailnum	flight	origin	dest	air_time	distance	cancelled	diverted	hour	minute	time_hour
2013	10	1	655	700	-5	1508	1530	-22	AA	N5DF	22	LAX	BOS	286	2611	0	0	7	0	2013-10-01 07:00:00
2013	10	1	704	710	-6	1006	1035	-29	AA	N3GP	25	BOS	LAX	337	2611	0	0	7	10	2013-10-01 07:10:00
2013	10	1	707	715	-8	1000	1030	-30	VX	N837	73	BOS	LAX	329	2611	0	0	7	15	2013-10-01 07:15:00
2013	10	1	755	800	-5	1611	1626	-15	B6	N605	28	LAX	BOS	300	2611	0	0	8	0	2013-10-01 08:00:00
2013	10	1	800	800	0	1054	1117	-23	B6	N612	87	BOS	LAX	331	2611	0	0	8	0	2013-10-01 08:00:00

year	month	day	dep_time	sched_dep_time	dep_delay	arr_time	sched_arr_time	arr_delay	carrier	flight	tailnum	origin	dest	air_time	distance	cancelled	diverted	hour	minute	time_hour
2013	10	1	804	800	4	1630	1630	0	VX	635	N634VA	LAX	BOS	310	2611	0	0	8	0	2013-10-01 08:00:00
2013	10	1	838	844	-6	1706	1716	-10	UA	1552	N515UA	LAX	BOS	299	2611	0	0	8	44	2013-10-01 08:44:00
2013	10	1	1108	1110	-2	1421	1425	-4	VX	635	N633VA	BOS	LAX	347	2611	0	0	11	10	2013-10-01 11:10:00
2013	10	1	1202	1205	-3	2015	2040	-25	AA	302	N3JUAA	LAX	BOS	291	2611	0	0	12	5	2013-10-01 12:05:00
2013	10	1	1208	1209	-1	2016	2035	-19	B6	488	N612JB	LAX	BOS	291	2611	0	0	12	9	2013-10-01 12:09:00

`LIMIT` restricts the number of rows returned. This is useful for previewing data without pulling the entire table from the server.

```
-- select all columns and limit to top 2 rows

SELECT *
FROM flights
LIMIT 2;
```

Table 2.4: 2 records

year	month	day	dep_time	sched_dep_time	dep_delay	arr_time	sched_arr_time	arr_delay	carrier	flight	tailnum	origin	dest	air_time	distance	cancelled	diverted	hour	minute	time_hour
2013	10	1	2	10	-8	453	505	-12	AA	2012	N201AA	LAX	DFW	249	1235	0	0	0	10	2013-10-01 00:10:00

year	month	day	dep_time	sched_dep_time	dep_delay	arr_time	sched_arr_time	arr_delay	carrier	flight	tailnum	origin	dest	air_time	distance	cancelled	diverted	hour	minute	time_hour
2013	10	1	4	2359	5	730	729	1	FL	1	N344AT	LGA	SFO	247	2139	0	0	23	59	2013-10-01 23:59:00

`GROUP BY` partitions rows into groups sharing the same value of one or more columns, and aggregate functions like `COUNT()` are applied within each group. `ORDER BY` sorts the result; the default sort order is ascending (use `ORDER BY num_flights DESC` for descending order).

```
-- select year, and the computing num_flights from the flights table
-- grouping by year
-- ordering by the num_flights

SELECT year,
       COUNT(*) AS num_flights
FROM flights
GROUP BY year
ORDER BY num_flights;
```

Table 2.5: 3 records

year	num_flights
2015	5819079
2014	5819811
2013	6369482

i Written order of `SQL` clauses

Queries in `SQL` start with the **SELECT** keyword and consist of several **clauses**, which have to be written in this order:

- **SELECT** allows you to list the columns, or functions operating on columns, that you want to retrieve.
- **FROM** specifies the table where the data are.
- **JOIN** allows you to stitch together two or more tables using a key.
- **WHERE** allows you to extract rows according to some criteria.

- **GROUP BY** allows you to aggregate the records according to some shared value.
- **HAVING** is like a `WHERE` clause that operates on the result set—not the original rows themselves.
- **ORDER BY** orders the rows of the result set.
- **LIMIT** restricts the number of rows in the output.

The following queries explore the `flights` table, starting with basic retrieval and building toward more complex aggregations.

```
DESCRIBE flights;
```

Table 2.6: Displaying records 1 - 10

Field	Type	Null	Key	Default	Extra
year	smallint(4)	YES		NA	
month	smallint(2)	YES		NA	
day	smallint(2)	YES		NA	
dep_time	smallint(4)	YES		NA	
sched_dep_time	smallint(4)	YES		NA	
dep_delay	smallint(4)	YES		NA	
arr_time	smallint(4)	YES		NA	
sched_arr_time	smallint(4)	YES		NA	
arr_delay	smallint(4)	YES		NA	
carrier	varchar(2)	NO			

2.4.1 query 1

`SELECT *` retrieves all columns. Combined with `LIMIT`, this is a quick way to preview the contents of a table without loading all rows.

```
-- display the first ten rows of the flights table

SELECT *
FROM flights
LIMIT 10;
```

Table 2.7: Displaying records 1 - 10

year	month	day	dep_time	dep_delay	arr_time	arr_delay	carrier	tailnum	flight	origin	dest	air_time	distance	cancelled	diverted	hour	minute	time_hour
2013	10	1	2	10	-8	453	505	-12	AA	N201AA	2400	LAX	DFW	149	1235	0	0	0	10	2013-10-01 00:10:00
2013	10	1	4	2359	5	730	729	1	FL	N344AT	7410	SFO	ATL	247	2139	0	0	23	59	2013-10-01 23:59:00
2013	10	1	11	15	-4	528	530	-2	AA	N3KMAA	1528	SFO	DFW	182	1464	0	0	0	15	2013-10-01 00:15:00
2013	10	1	14	2355	19	544	540	4	AA	N3EHAA	2392	SEA	ORD	191	1721	0	0	23	55	2013-10-01 23:55:00
2013	10	1	16	17	-1	515	525	-10	UA	N38451	1714	LAX	IAH	157	1379	0	0	0	17	2013-10-01 00:17:00
2013	10	1	22	20	2	552	554	-2	UA	N45829	1791	SFO	IAH	188	1635	0	0	0	20	2013-10-01 00:20:00
2013	10	1	29	35	-6	808	816	-8	US	N55143	1736	LAX	CLT	256	2125	0	0	0	35	2013-10-01 00:35:00
2013	10	1	29	35	-6	449	458	-9	AS	N402AS	108	ANC	SEA	181	1448	0	0	0	35	2013-10-01 00:35:00
2013	10	1	31	30	1	519	538	-19	DL	N596DN	1896	SEA	MSP	147	1399	0	0	0	30	2013-10-01 00:30:00

year	month	day	dep_time	sched_dep_time	dep_delay	arr_time	sched_arr_time	arr_delay	carrier	flight	tailnum	origin	dest	air_time	distance	cancelled	diverted	hour	minute	time_hour
2013	10	1	32	33	-1	557	606	-9	DL	1406	N687DA	LAX	MSP	183	1535	0	0	0	33	2013-10-01 00:33:00

Essentially, `LIMIT` takes two arguments - the first skips a specific number of rows (offset), the second returns the specified row count after skipping.

```
-- skip the first 18,008,362 rows and return the next 10
-- how did we find out the total number of rows?

SELECT *
FROM flights
LIMIT 18008362, 10;
```

Table 2.8: Displaying records 1 - 10

year	month	day	dep_time	sched_dep_time	dep_delay	arr_time	sched_arr_time	arr_delay	carrier	flight	tailnum	origin	dest	air_time	distance	cancelled	diverted	hour	minute	time_hour
2015	9	30	0	930	0	0	1100	0	WN	2608	N653SW	PHX	LAX	0	370	1	0	9	30	2015-09-30 09:30:00
2015	9	30	0	1100	0	0	1340	0	WN	2652	N402WN	SEA	SNA	0	978	1	0	11	0	2015-09-30 11:00:00
2015	9	30	0	735	0	0	1025	0	WN	2652	N402WN	SNA	SEA	0	978	1	0	7	35	2015-09-30 07:35:00
2015	9	30	0	600	0	0	850	0	NK	626	N605ZK	ORD	ATL	0	606	1	0	6	0	2015-09-30 06:00:00

year	month	day	dep_time	sched_dep_time	dep_delay	arr_time	sched_arr_time	arr_delay	carrier	tailnum	flight	origin	dest	air_time	distance	cancelled	diverted	hour	minute	time_hour
2015	9	30	0	940	0	0	1050	0	NK	N603[illegible]	363	LAX	ATL	IAH	0	689	1	0	9	40	2015-09-30 09:40:00
2015	9	30	0	1955	0	0	2256	0	NK	N640[illegible]	779	LGA	FLL	0	1076	1	0	19	55	2015-09-30 19:55:00
2015	9	30	0	1054	0	0	1228	0	EV	N916[illegible]	5222	ORD	ATL	0	399	1	0	10	54	2015-09-30 10:54:00
2015	9	30	0	1950	0	0	2035	0	OO	N875[illegible]	2856	PHX	FLG	0	119	1	0	19	50	2015-09-30 19:50:00
2015	9	30	0	620	0	0	804	0	OO	N875[illegible]	2889	SBP	PHX	0	509	1	0	6	20	2015-09-30 06:20:00
2015	9	30	0	1145	0	0	2025	0	UA	NA	1766	SFO	EWR	0	2565	1	0	11	45	2015-09-30 11:45:00

2.4.2 query 2

Rather than retrieving all columns, we can name specific columns in the `SELECT` clause. This reduces the amount of data transferred from the server and makes results easier to interpret.

```
-- display year, month, day, origin, dep_time, sched_dep_time, dep_delay for the first ten r

SELECT year, month, day, origin, dep_time, sched_dep_time, dep_delay
FROM flights
LIMIT 10;
```

Table 2.9: Displaying records 1 - 10

year	month	day	origin	dep_time	sched_dep_time	dep_delay
2013	10	1	LAX	2	10	-8
2013	10	1	SFO	4	2359	5
2013	10	1	SFO	11	15	-4
2013	10	1	SEA	14	2355	19
2013	10	1	LAX	16	17	-1
2013	10	1	SFO	22	20	2
2013	10	1	LAX	29	35	-6
2013	10	1	ANC	29	35	-6
2013	10	1	SEA	31	30	1
2013	10	1	LAX	32	33	-1

We can also concatenate strings and cast them into date formats natively.

```
SELECT STR_TO_DATE(CONCAT(year, '-', month, '-', day), '%Y-%m-%d') AS 'date', origin, dep_ti
FROM flights
LIMIT 10;
```

Table 2.10: Displaying records 1 - 10

date	origin	dep_time	sched_dep_time	dep_delay
2013-10-01	LAX	2	10	-8
2013-10-01	SFO	4	2359	5
2013-10-01	SFO	11	15	-4
2013-10-01	SEA	14	2355	19
2013-10-01	LAX	16	17	-1
2013-10-01	SFO	22	20	2
2013-10-01	LAX	29	35	-6
2013-10-01	ANC	29	35	-6
2013-10-01	SEA	31	30	1
2013-10-01	LAX	32	33	-1

2.4.3 query 3

`COUNT(*)` counts every row in the table regardless of `NULL` values in individual columns. Aliasing the result with `AS` gives the output column a descriptive name.

```sql
-- count the total number of rows in the flights table

SELECT COUNT(*) AS total_rows
FROM flights;
```

Table 2.11: 1 records

total_rows
18008372

`COUNT(*)` counts *rows*, but we often want to count **unique values** instead — for example, how many different carriers are in this table. In `R`, `n_distinct()` does this; in `SQL`, we pair `COUNT()` with the `DISTINCT` keyword.

```sql
SELECT COUNT(DISTINCT carrier) AS num_carriers
FROM flights;
```

Table 2.12: 1 records

num_carriers
17

2.4.4 query 4

`WHERE` filters rows before any aggregation takes place. Here we restrict to flights from ATW to ORD in 2013, combining multiple conditions with `AND`.

```sql
-- extract all info about flights that flew from Appleton (ATW) to Chicago (ORD) in 2013

SELECT *
FROM flights
WHERE origin = 'ATW' AND dest = 'ORD' AND year = 2013;
```

Table 2.13: Displaying records 1 - 10

year	month	day	dep_time	sched_dep_time	dep_delay	arr_time	sched_arr_time	arr_delay	carrier	flight	tailnum	origin	dest	air_time	distance	cancelled	diverted	hour	minute	time_hour
2013	10	1	543	545	-2	633	640	-7	EV	4112	N14982	ATW	ORD	35	160	0	0	5	45	2013-10-01 05:45:00
2013	10	1	927	928	-1	1027	1023	4	OO	5819	N917EV	ATW	ORD	37	160	0	0	9	28	2013-10-01 09:28:00
2013	10	1	1137	1137	0	1240	1230	10	EV	4112	N14952	ATW	ORD	39	160	0	0	11	37	2013-10-01 11:37:00
2013	10	1	1312	1322	-10	1404	1417	-13	EV	4112	N15561	ATW	ORD	36	160	0	0	13	22	2013-10-01 13:22:00
2013	10	1	1619	1627	-8	1728	1724	4	EV	4112	N12922	ATW	ORD	33	160	0	0	16	27	2013-10-01 16:27:00
2013	10	1	1818	1823	-5	1920	1922	-2	EV	4112	N15542	ATW	ORD	33	160	0	0	18	23	2013-10-01 18:23:00
2013	10	2	542	545	-3	631	640	-9	EV	4112	N14982	ATW	ORD	33	160	0	0	5	45	2013-10-02 05:45:00
2013	10	2	923	928	-5	1018	1023	-5	OO	5819	N920EV	ATW	ORD	37	160	0	0	9	28	2013-10-02 09:28:00
2013	10	2	1127	1137	-10	1212	1230	-18	EV	4112	N13952	ATW	ORD	33	160	0	0	11	37	2013-10-02 11:37:00

year	month	day	dep_time	sched_dep_time	dep_delay	arr_time	sched_arr_time	arr_delay	carrier	tailnum	flight	origin	dest	air_time	distance	cancelled	diverted	hour	minute	time_hour
2013	10	2	1307	1322	-15	1353	1417	-24	EV	N13991	6171	ATW	ORD	88	160	0	0	13	22	2013-10-02 13:22:00

Logical operators `AND` and `OR`

Logical operators in `SQL` evaluate in a strict order: `AND` is evaluated before `OR`.
Imagine we are looking at our `flights` table and we want to find all flights in the year 2013 that either departed from Appleton (ATW) or arrived in Chicago (ORD).
It is tempting to write the query exactly as we speak it:

```
SELECT COUNT(*) AS num_flights
FROM flights
WHERE year = 2013 AND origin = 'ATW' OR dest = 'ORD';
```

Table 2.14: 1 records

num_flights
910681

Because `AND` has higher precedence, the `SQL` engine secretly groups the query like this:

```
SELECT COUNT(*)
FROM flights
WHERE (year = 2013 AND origin = 'ATW') OR dest = 'ORD';
```

Table 2.15: 1 records

COUNT(*)
910681

What this actually returns:

1. Every flight that departed from ATW in 2013.
2. PLUS every single flight that landed in ORD, regardless of the year, and regardless of where it came from.

Instead of returning a small subset of specific 2013 flights, our query returns millions of rows, pulling in ORD flights from 2001, 2010, etc.
To fix this, we use parentheses to force the database to evaluate the `OR` condition as a single, combined logical unit before it evaluates the `AND` condition.

```
SELECT COUNT(*) AS num_flights
FROM flights
WHERE year = 2013 AND (origin = 'ATW' OR dest = 'ORD');
```

Table 2.16: 1 records

num_flights
309790

How `SQL` evaluates this:

1. First, it looks inside the parentheses: "Is the origin ATW, `OR` is the destination ORD?" (Returns True or False for each row).
2. Then, it evaluates the `AND`: "Is the year 2013 `AND` did the previous step return True?"

By wrapping the `OR` statements in brackets, we ensure that the `year = 2013` condition acts as a strict filter applied to both the origin and the destination checks.
A Golden Rule for `SQL`:
Whenever you have a `WHERE` clause that mixes `AND` and `OR`, always use parentheses. Even if you memorize the precedence rules, parentheses make your code instantly readable to your colleagues and future self, eliminating any ambiguity about your analytical intentions.

i `BETWEEN` vs `IN`

Notice the difference between the `BETWEEN` and `IN` operators.

```sql
-- number of flights that were delayed at departure by less than 10 minutes

SELECT COUNT(*) as num_rows
FROM flights
WHERE dep_delay BETWEEN 0 AND 10;
```

Table 2.17: 1 records

num_rows
4140140

```sql
-- number of flights that were delayed by either 0 or 10 minutes

SELECT COUNT(*) as num_rows
FROM flights
WHERE dep_delay IN (0,10);
```

Table 2.18: 1 records

num_rows
1471502

2.4.5 query 5

Adding `GROUP BY year` counts the number of flights that operated on the ATW–ORD route per year.

`ORDER BY year` sorts the results chronologically.

```sql
-- count the number of flights from Appleton (ATW) for each year

SELECT year, COUNT(*) AS num_flights
FROM flights
WHERE origin = 'ATW' AND dest = 'ORD'
GROUP BY year
ORDER BY year;
```

Table 2.19: 3 records

year	num_flights
2013	1838
2014	1647
2015	1570

We can group by multiple variables by separating them with commas. Grouping by both `carrier` and `year` lets us see how many flights each airline operated on the ATW–ORD route per year.

```
-- which carriers flew from Appleton (ATW) to Chicago (ORD)
-- how many flights did they do per year

SELECT carrier, year, COUNT(*) AS num_flights
FROM flights
WHERE origin = 'ATW' AND dest = 'ORD'
GROUP BY carrier, year
ORDER BY carrier, year;
```

Table 2.20: 6 records

carrier	year	num_flights
EV	2013	1634
EV	2014	1135
EV	2015	1297
OO	2013	204
OO	2014	512
OO	2015	273

2.4.6 query 6

`MAX(dep_time)` within each carrier group returns the latest recorded departure time for that carrier on the ATW–ORD route.

Note that departure times in this table are stored as integers in `HHMM` format (e.g., 1435 = 2:35 PM).

```sql
-- for flights that flew from Appleton (ATW) to Chicago (ORD)
-- what was the latest flight for each carrier

SELECT carrier, MAX(dep_time) AS latest_dep_time
FROM flights
WHERE origin = 'ATW' AND dest = 'ORD'
GROUP BY carrier
LIMIT 1;
```

Table 2.21: 1 records

carrier	latest_dep_time
EV	2309

2.4.7 query 7

Understanding the difference between `WHERE` and `HAVING` is one of the most important hurdles in learning `SQL`. Both act as filters, but they operate at different stages of the execution pipeline.

`WHERE` filters records in the original dataset before any grouping occurs. `HAVING` filters the results of our aggregations after the `GROUP BY` clause has executed.

First, let us find the destination with the lowest average arrival delay from Appleton, using `WHERE` to filter the origin.

```sql
-- for flights that flew from Appleton (ATW)
-- which destination had the lowest average arrival delay time

SELECT dest, COUNT(*) AS num_flights, AVG(arr_delay) AS avg_arr_delay
FROM flights
WHERE origin = 'ATW'
GROUP BY dest
ORDER BY avg_arr_delay;
```

Table 2.22: 4 records

dest	num_flights	avg_arr_delay
DTW	2183	-0.6949
MSP	1920	0.9250
ATL	1756	6.5017

dest	num_flights	avg_arr_delay
ORD	5055	13.9937

Suppose we want to restrict our analysis to robust routes — say, those with at least 2,000 flights. We cannot put `num_flights >= 2000` in the `WHERE` clause because `num_flights` does not exist in the original table; it is an aggregated calculation. Therefore, we must use `HAVING`.

```
-- for flights that flew from Appleton (ATW)
-- consider only destinations with at least 2000 flights from Appleton (ATW)
-- which destination had the lowest average arrival delay time

SELECT dest, COUNT(*) AS num_flights, AVG(arr_delay) AS avg_arr_delay
FROM flights
WHERE origin = 'ATW'
GROUP BY dest
HAVING num_flights >= 2000
ORDER BY avg_arr_delay;
```

Table 2.23: 2 records

dest	num_flights	avg_arr_delay
DTW	2183	-0.6949
ORD	5055	13.9937

`HAVING` filters groups **after** aggregation, unlike `WHERE` which filters rows **before** aggregation.

2.4.8 query 8

We now have all the pieces to write a complete query using every clause introduced in this chapter. Notice how each clause does a distinct job, and how the order we write them in matches the order listed at the start of this section: we filter raw rows first (`WHERE`), then aggregate (`GROUP BY`), then filter the aggregated results (`HAVING`), then sort (`ORDER BY`), and finally restrict the output size (`LIMIT`). Try reading the query out loud, clause by clause — it should read almost like a sentence describing the question we are asking of the data.

```
-- for flights out of Appleton (ATW) in 2013 or later
-- consider only destinations with at least 500 flights
-- find the 5 destinations with the highest average departure delay
```

```sql
SELECT dest,
       COUNT(*) AS num_flights,
       AVG(dep_delay) AS avg_dep_delay
FROM flights
WHERE origin = 'ATW' AND year >= 2013
GROUP BY dest
HAVING num_flights >= 500
ORDER BY avg_dep_delay DESC
LIMIT 5;
```

Table 2.24: 4 records

dest	num_flights	avg_dep_delay
ORD	5055	11.7658
ATL	1756	9.4129
DTW	2183	3.5456
MSP	1920	2.2849

Which approach to use?

Having now seen all three approaches to writing `SQL` from within `R`, a natural question is: which one should we actually use? There is no single right answer, but the following rough guide is a useful starting point:

- **Use `dbplyr` (Part 1)** when your query is a fairly standard `dplyr` pipeline — filtering, grouping, summarizing — and you want your code to stay readable and consistent with the rest of a `tidyverse`-based analysis. This is the best default for most exploratory work.
- **Use `DBI::dbGetQuery()` (Part 2)** when you need a `SQL` feature `dbplyr` cannot translate (such as window functions), but the query is short enough to comfortably live as a string inside an `R` script or function.
- **Use native `{sql}` code chunks (Part 3)** when writing long, complex, or highly `SQL`-specific queries — window functions, subqueries, or multi-table joins — where syntax highlighting and formatting readability matter, or when you want a query that could be copy-pasted directly into a standalone database client.

In practice, many real analyses mix all three approaches within the same project, choosing whichever tool best fits the query at hand.

2.5 calling SQL generated datasets into R for data viz

Our ultimate workflow often involves doing the heavy lifting (filtering and aggregating) in SQL, and then pulling the summarized dataset into R to leverage packages like ggplot2. We have three ways to save the output of a SQL query into an R object:

2.5.1 Method 1: using DBI::dbGetQuery()

```
df1 <- DBI::dbGetQuery(con_air, "SELECT * FROM flights LIMIT 10")

# always check the number of rows
```

2.5.2 Method 2: using dbplyr with raw SQL

```
df2 <- dplyr::tbl(con_air, dplyr::sql("SELECT * FROM flights LIMIT 10")) |> dplyr::collect()

# doesn't need a semicolon
```

2.5.3 Method 3: Quarto chunk options

We can write a native SQL chunk and use the `--| output.var: "df"` code chunk option to assign the result to an object in our R environment.

```
SELECT * FROM flights LIMIT 10;
```

All three methods are valid. We select the one that best fits our workflow and the complexity of the query we need to execute.

Finally, always remember to disconnect your connection to the server.

```
# terminate the SQL connection

DBI::dbDisconnect(con_air, shutdown = TRUE)
```

3 Joins, Unions, and Subqueries

3.1 load packages and connect to database server

```
library(tidyverse)
library(mdsr)
library(DBI)
library(RMariaDB)
library(dbplyr)
```

```
# access airlines database

con_air <- mdsr::dbConnect_scidb("airlines")

DBI::dbListTables(con_air)   # list tables in database
```

```
# access tables from the airlines database

flights <- dplyr::tbl(con_air, "flights")

carriers <- dplyr::tbl(con_air, "carriers")

airports <- dplyr::tbl(con_air, "airports")
```

```
DESCRIBE flights;
```

```
SELECT * FROM flights LIMIT 10;
```

```
DESCRIBE carriers;
```

```
SELECT * FROM carriers LIMIT 100;
```

```
DESCRIBE airports;
```

```
SELECT * FROM airports LIMIT 100;
```

3.2 Joins

In general, there are four pieces of information that you need to specify in order to join two tables:

- name of the first table
- type of join
- name of the second table
- condition(s) of the join

The following join types are available in MySQL dialect of SQL:

- **JOIN**
- **LEFT JOIN**
- **RIGHT JOIN**
- **CROSS JOIN**

```
-- access airport names for flights that flew out of Appleton in 2013
```

```
-- access both airport and carrier names for flights that flew out of Appleton in 2013
```

```
-- using LEFT JOIN
```

3.3 Unions

```
-- extract all flights that flew from ATW to MSP and from JFK to ORD on April 14, 2013
```

3.4 Subqueries

```
-- count the number of flights that satisfy the criteria above
```

```
# terminate the SQL connection

DBI::dbDisconnect(con_air, shutdown = TRUE)
```

4 Creating Databases

4.1 load packages and connect to database server

```
library(tidyverse)
library(mdsr)
library(DBI)
library(RMariaDB)
library(dbplyr)
library(duckdb)
```

As an example, we will consider the Saturday Night Live datasets available on the **snldb** GitHub repo. Data is scraped from SNL archives and http://www.imdb.com/title/tt0072562 by Hendrik Hilleckes and Colin Morris.

```
con_snl <- DBI::dbConnect(duckdb::duckdb(), dbdir = "snl_db")
```

```
SHOW TABLES;
```

4.2 accessing data (csv files) from GitHub (or, elsewhere)

Notice that there are eleven **.csv** files available in the **output** folder. Let's look at the **casts** dataset in R, so that we understand the data we want to load into the database.

```
# read in the data

casts <- readr::read_csv("")
```

```
glimpse(casts)
```

```
summary(casts)
```

```
# save the dataset as a .csv file on your (local) computer

write_csv(casts, "casts.csv")
```

4.3 data types in SQL

Consult data types for details.

4.4 importing data

Once the database is set up, you will be ready to import .csv files into the database as tables. Importing .csv files as tables requires a series of steps:

- a **USE** statement that ensures we are in the right database.
- a series of **DROP TABLE** statements that drop any old tables with the same names as the ones we are going to create.
- a series of **CREATE TABLE** statements that specify the table structures and field types.
- a series of **COPY** (or **LOAD DATA** for MySQL) statements that read the data from the .csv files into the appropriate tables.

```
USE snl_db;
```

```
DROP TABLE IF EXISTS casts;
```

```
CREATE TABLE casts (
  aid VARCHAR(255) NOT NULL DEFAULT ' ',
  sid SMALLINT NOT NULL DEFAULT 0,
  featured BOOLEAN NOT NULL DEFAULT FALSE,  /* use TINYINT(1) in MySQL instead of BOOLEAN) *,
  first_epid INTEGER DEFAULT NULL,
  last_epid INTEGER DEFAULT NULL,
  update_anchor BOOLEAN NOT NULL DEFAULT FALSE,
  n_episodes SMALLINT NOT NULL DEFAULT 0,
  season_fraction DOUBLE NOT NULL DEFAULT 0,
  PRIMARY KEY (sid, aid)
);
```

```
COPY casts FROM 'casts.csv' (FORMAT CSV, HEADER, NULL 'NA');
```

4.5 checks

```
SHOW DATABASES;
```

```
SHOW TABLES;
```

```
DESCRIBE casts;
```

```
SELECT * FROM casts LIMIT 10;
```

4.6 shortcut (not accepted for Project 2)

```
# read in the data

episodes <- readr::read_csv("")

glimpse(episodes)
```

```
# save the dataset as a .csv file on your (local) computer

write_csv(episodes, "episodes.csv")
```

```
duckdb_read_csv(con = con_snl, name = "episodes", files = "episodes.csv")
```

```
SHOW TABLES;
```

```
DESCRIBE episodes;
```

```
SELECT * FROM episodes LIMIT 10;
```

```
# terminate the SQL connection

DBI::dbDisconnect(con_snl, shutdown = TRUE)
```

Part II

Text Analysis

5 Case Study of Shakepearse's Macbeth using Regular Expressions

5.1 resources

- Text Mining with R
- Modern Data Science with R (3e) - Chapter 19

5.2 packages

```
library(tidyverse)
```

5.3 data

Our data comes from Project Gutenberg, a large repository of free eBooks. We will be working with the text of Shakespeare's play Macbeth, which is available at https://www.gutenberg.org/cache/epub/1129/pg1129.txt. The text is in the public domain, so we are free to use it for our analysis.

```
library(gutenbergr)

data("gutenberg_metadata")

macbeth_all <- gutenberg_metadata |>
  filter(str_detect(title, "Macbeth"))

book <- gutenberg_download(1129, meta_fields = "title")
```

For our case study, we will be working with the raw text of the play, which is available in the `mdsr` package.

```
library(mdsr)

data(Macbeth_raw)

?Macbeth_raw
```

```
macbeth <- Macbeth_raw |>
  str_split("\r\n") |>
  pluck(1)

length(macbeth)

macbeth[300:310]
```

5.4 strings and regular expressions

```
# how many times does the character Macbeth speak in the play?

macbeth_lines <- macbeth |>
  str_subset("  MACBETH")

length(macbeth_lines)

head(macbeth_lines)
```

```
macbeth |>
  str_which("  MACBETH")
```

5.5 about regular expressions grammar

5.5.1 metacharacter

```
macbeth |>
  str_subset("MAC.") |>
  head()
```

```
macbeth |>
  str_subset("MACBETH\\.") |>
  head()
```

5.5.2 character sets

```
macbeth |>
  str_subset("MAC[B-Z]") |>
  head()
```

5.5.3 alternation

```
macbeth |>
  str_subset("MAC(B|D)") |>
  head()
```

5.5.4 anchors (power-dollar)

```
macbeth |>
  str_subset("^  MAC[B-Z]") |>
  head()
```

```
macbeth |>
  str_subset("\\.$") |>
  head()
```

5.5.5 repetitions

```
# ?

macbeth |>
  str_subset("^ ?MAC[B-Z]") |>
  head()
```

```
# *

macbeth |>
  str_subset("^ *MAC[B-Z]") |>
  head()
```

```
# +

macbeth |>
  str_subset("^ +MAC[B-Z]") |>
  head()
```

5.6 a case study - life and death in Macbeth and speaking patterns

```
macbeth_chars <- tribble(
  ~name, ~regexp,
  "Macbeth", "  MACBETH\\.",
  "Lady Macbeth", "  LADY MACBETH\\.",
  "Banquo", "  BANQUO\\.",
  "Duncan", "  DUNCAN\\.",
) |>
  mutate(speaks = purrr::map(regexp, str_detect, string = macbeth))
```

```
speaker_freq <- macbeth_chars |>
  unnest(cols = speaks) |>
  mutate(
    line = rep(1:length(macbeth), 4),
    speaks = as.numeric(speaks)
  ) |>
  filter(line > 218 & line < 3172)

glimpse(speaker_freq)
```

```
acts <- tibble(
  line = str_which(macbeth, "^ACT [I|V]+"),
  line_text = str_subset(macbeth, "^ACT [I|V]+"),
  labels = str_extract(line_text, "^ACT [I|V]+")
)
```

```r
ggplot(data = speaker_freq, aes(x = line, y = speaks)) +
  geom_smooth(
    aes(color = name), method = "loess",
    se = FALSE, span = 0.4
  ) +
  geom_vline(
    data = acts,
    aes(xintercept = line),
    color = "darkgray", lty = 3
  ) +
  geom_text(
    data = acts,
    aes(y = 0.085, label = labels),
    hjust = "left", color = "darkgray"
  ) +
  ylim(c(0, NA)) +
  xlab("Line Number") +
  ylab("Proportion of Speeches") +
  scale_color_colorblind()
```

6 Case Study of Donald Trump's tweets during the 2016 US presidential election

6.1 resources

- Trump Twitter Archive
- Todd Vaziri's Tweet
- David Robinson's Blog Post
- Text Mining With R

6.2 packages

```
library(tidyverse)
library(scales)
library(tidytext)
library(textdata)
library(dslabs)
```

6.3 data

Our data come from the *dslabs* package.

```
data("trump_tweets")

?trump_tweets

glimpse(trump_tweets)
```

```
range(trump_tweets$created_at)
```

```
trump_tweets$text[16413]
```

```
trump_tweets |>
  count(source) |>
  arrange(desc(n))
```

For our case study, we are interested in what happened during the 2016 campaign, so for this analysis we will focus on what was tweeted between the day Trump announced his campaign and election day. Also, we only keep tweets from Android or iPhone, and filter out retweets.

```
campaign_tweets <- trump_tweets |>
  filter(source %in% paste("Twitter for", c("Android", "iPhone")) &
           created_at >= ymd("2015-06-17") &
           created_at < ymd("2016-11-08")) |>
  mutate(source = str_remove(source, pattern = "Twitter for ")) |>
  filter(!is_retweet) |>
  arrange(created_at) |>
  as_tibble()
```

```
campaign_tweets |>
  count(source)
```

6.4 some initial exploration

We explore the possibility that two different groups were tweeting from these devices.

```
campaign_tweets |>
  mutate(hour = hour(with_tz(created_at, "EST"))) |>
  count(source, hour)
```

```
android_vs_iphone_hourly_props <- campaign_tweets |>
  mutate(hour = hour(with_tz(created_at, "EST"))) |>
  count(source, hour) |>
  group_by(source) |>
  mutate(percent = n / sum(n))

android_vs_iphone_hourly_props
```

```
android_vs_iphone_hourly_props |>
  ggplot(aes(x = hour, y = percent, color = source)) +
  geom_line() +
  geom_point() +
  scale_y_continuous(labels = percent_format()) +
  labs(x = "Hour of day (EST)", y = "% of tweets", color = "")
```

6.5 tokens

```
i <- 3008

campaign_tweets$text[i]
```

```
campaign_tweets[i,] |>
  tidytext::unnest_tokens(output = word, input = text, token = "words") |>
  dplyr::pull(word)
```

```
links_to_pics <- "(https://t.co/[A-Za-z\\d]+)|(http://t.co/[A-Za-z\\d]+)"

campaign_tweets[i,] |>
  mutate(text = str_remove_all(text, pattern = links_to_pics)) |>
  tidytext::unnest_tokens(output = word, input = text, token = "words") |>
  dplyr::pull(word)
```

```
tweet_words <- campaign_tweets |>
  mutate(text = str_remove_all(text, pattern = links_to_pics))  |>
  tidytext::unnest_tokens(output = word, input = text, token = "words")
```

```
tweet_words |>
  count(word) |>
  arrange(desc(n))
```

6.6 stop words

```r
data(stop_words)

?stop_words
```

```r
tweet_words <- campaign_tweets |>
  mutate(text = str_remove_all(text, pattern = links_to_pics))  |>
  tidytext::unnest_tokens(output = word, input = text, token = "words") |>
  filter(!(word %in% stop_words$word))

tweet_words |>
  count(word)
```

```r
tweet_words <- campaign_tweets |>
  mutate(text = str_remove_all(text, links_to_pics))  |>
  tidytext::unnest_tokens(output = word, input = text, token = "words") |>
  filter(!(word %in% stop_words$word)) |>
  filter(!(str_detect(word, pattern = "^\\d+")))

tweet_words |>
  count(word)

# |>
#   arrange(desc(n))
```

6.7 further exploration

```r
android_vs_iphone <- tweet_words |>
  count(word, source)

android_vs_iphone
```

```r
android_vs_iphone <- tweet_words |>
  count(word, source) |>
  pivot_wider(names_from = "source", values_from = "n", values_fill = 0)

android_vs_iphone
```

```
android_vs_iphone <- tweet_words |>
  count(word, source) |>
  pivot_wider(names_from = "source", values_from = "n", values_fill = 0) |>
  mutate(p_a = (Android + 0.5)/(sum(Android) + 0.5),
         p_i = (iPhone + 0.5)/(sum(iPhone) + 0.5),
         ratio = p_a / p_i)

android_vs_iphone
```

```
# for words appearing at least 100 times in total, here are the ten highest percent differenc

top_10_android <- android_vs_iphone |>
  filter(Android + iPhone >= 100) |>
  arrange(desc(ratio)) |>
  slice_head(n = 10)

top_10_android
```

```
# and the top ten for iPhone

top_10_iphone <- android_vs_iphone |>
  filter(Android + iPhone >= 100) |>
  arrange(ratio)|>
  slice_head(n = 10)

top_10_iphone
```

```
top_10_android |>
  rbind(top_10_iphone) |>
  dplyr::select(word, iPhone, Android, ratio) |>
  mutate(log_ratio = log(ratio)) |>
  mutate(device = rep(c("Android", "iPhone"), each = 10)) |>
  ggplot() +
  geom_col(aes(x = log_ratio, y = fct_reorder(word, log_ratio), fill = device))
```

6.8 sentiment analysis

```
tidytext::get_sentiments("bing")
```

```
tidytext::get_sentiments("afinn")

tidytext::get_sentiments("nrc")
```

```
nrc <- tidytext::get_sentiments("nrc")

nrc |> count(sentiment)
```

```
tweet_words |>
  left_join(nrc, by = "word") |>
  dplyr::select(source, word, sentiment) |>
  sample_n(5)
```

```
sentiment_counts <- tweet_words |>
  left_join(nrc, by = "word", relationship = "many-to-many") |>
  count(source, sentiment)

sentiment_counts
```

```
sentiment_counts <- tweet_words |>
  left_join(nrc, by = "word", relationship = "many-to-many") |>
  count(source, sentiment) |>
  pivot_wider(names_from = "source", values_from = "n") |>
  mutate(sentiment = replace_na(sentiment, replace = "none"))

sentiment_counts
```

```
sentiment_counts <- sentiment_counts |>
  mutate(p_a = Android/sum(Android),
         p_i = iPhone/sum(iPhone),
         ratio = p_a/p_i) |>
  arrange(desc(ratio))

sentiment_counts
```

```
ordered_levels <- sentiment_counts$sentiment

android_vs_iphone |>
  filter(Android + iPhone >= 10) |>
  left_join(nrc, by = "word") |>
```

```
  mutate(sentiment = factor(sentiment, levels = sentiment_counts$sentiment)) |>
  group_by(sentiment) |>
  slice_max(abs(log(ratio)), n = 10) |>
  ungroup() |>
  mutate(word = reorder(word, -ratio)) |>
  ggplot(aes(word, ratio, fill = ratio < 1)) +
  geom_col(show.legend = FALSE) +
  scale_y_log10() +
  facet_wrap(~sentiment, scales = "free_x", nrow = 2) +
  theme(axis.text.x = element_text(angle = 90, hjust = 1))
```

7 Case Study of Data Science Papers from arXiv.org

7.1 resources

- arXiv.org

7.2 packages

```
library(tidyverse)
library(scales)
library(tidytext)
library(textdata)
library(mdsr)
```

7.3 data

We can import data using the **aRxiv** package.

```
library(aRxiv)

df <- arxiv_search(
  query = 'ti:"Data Science"',
  limit = 20000,
  batchsize = 100
)

glimpse(df)
```

For this case study we will use some already-collected data from the **mdsr** package.

```
library(mdsr)

data(DataSciencePapers)

?DataSciencePapers

glimpse(DataSciencePapers)
```

7.4 initial data exploration and cleaning

```
DataSciencePapers <- DataSciencePapers |>
  mutate(
    submitted = lubridate::ymd_hms(submitted),
    updated = lubridate::ymd_hms(updated)
  )

glimpse(DataSciencePapers)
```

```
mosaic::tally(~ year(submitted), data = DataSciencePapers)
```

```
DataSciencePapers |>
  filter(id == "1809.02408v2") |>
  glimpse()
```

```
DataSciencePapers |>
  group_by(primary_category) |>
  count()
```

```
DataSciencePapers <- DataSciencePapers |>
  mutate(field = str_extract(primary_category, pattern = "^[a-z,-]+"))

mosaic::tally(x = ~field, margins = TRUE, data = DataSciencePapers) |>
  sort()
```

7.5 corpora and tokenization

```
DataSciencePapers |>
  unnest_tokens(output = word, input = abstract, token = "words") |>
  count(id, word, sort = TRUE)
```

```
arxiv_words <- DataSciencePapers |>
  unnest_tokens(output = word, input = abstract, token = "words") |>
  anti_join(tidytext::get_stopwords(), by = "word")

arxiv_words |>
  count(id, word, sort = TRUE)
```

```
arxiv_abstracts <- arxiv_words |>
  group_by(id) |>
  summarize(abstract_clean = paste(word, collapse = " "))

arxiv_papers <- DataSciencePapers |>
  left_join(arxiv_abstracts, by = "id")
```

7.6 wordcloud

```
library(wordcloud)

set.seed(4302026)
arxiv_papers |>
  pull(abstract_clean) |>
  wordcloud(
    max.words = 40,
    scale = c(8, 1),
    colors = topo.colors(n = 30),
    random.color = TRUE
  )
```

7.7 bigrams

```
arxiv_bigrams <- arxiv_papers |>
  unnest_tokens(
```

```
    output = arxiv_bigram,
    input = abstract_clean,
    token = "ngrams",
    n = 2
  ) |>
  dplyr::select(arxiv_bigram, id)

arxiv_bigrams

arxiv_bigrams |>
  count(arxiv_bigram, sort = TRUE)
```

7.8 tf-idf and document term matrices

```
arxiv_words |>
  count(word) |>
  arrange(desc(n)) |>
  head()
```

```
tidy_DTM <- arxiv_words |>
  count(id, word) |>
  tidytext::bind_tf_idf(word, id, n)

tidy_DTM
```

```
tidy_DTM |>
  arrange(desc(tf)) |>
  head()
```

```
tidy_DTM |>
  arrange(desc(idf), desc(n)) |>
  head()
```

```
tidy_DTM |>
  filter(word == "wildfire")
```

```
arxiv_papers |>
  pull(abstract) |>
```

```
  str_subset("wildfire") |>
  strwrap() |>
  head()
```

```
tidy_DTM |>
  filter(word == "implications")
```

```
tidy_DTM |>
  filter(id == "1809.02408v2") |>
  arrange(desc(tf_idf)) |>
  head()
```

```
tm_DTM <- arxiv_words |>
  count(id, word) |>
  tidytext::cast_dtm(document = id, term = word, value = n)

tm_DTM
```

Part III

Geospatial Data

8 Recreating John Snow's 1854 map of the Broad Street cholera outbreak.

8.1 resources

- Modern Data Science with R (3e) - Chapter 17
- 1854 Broad Street cholera outbreak
- Shapefile

8.2 packages

```
library(tidyverse)
library(sf)
library(ggspatial)
library(maps)
library(mapproj)
```

8.3 data

```
root <- here::here("/cloud/project")  # where we stored these files

dsn <- fs::path(root, "SnowGIS_SHP")
```

```
list.files(dsn)
```

```
st_layers(dsn)
```

```r
CholeraDeaths <- st_read(dsn, layer = "Cholera_Deaths")
```

```r
class(CholeraDeaths)
```

```r
plot(CholeraDeaths["Count"])
```

```r
ggplot(CholeraDeaths) +
  geom_sf()
```

```r
ggplot(CholeraDeaths) +
  ggspatial::annotation_map_tile(type = "osm", zoomin = 0, progress = "none") +
  geom_sf(aes(size = Count), alpha = 0.7)
```

```r
# bounding boxes

st_bbox(CholeraDeaths)
```

8.4 projections

Top 10 World Map Projections

```r
maps::map(database = "world", projection = "mercator", wrap = TRUE)
```

```r
maps::map(database = "world", projection = "cylequalarea", param = 45, wrap = TRUE)
```

8.5 CRS - coordinate reference system

Coordinate Reference System - three formats that are common for storing information about the projection of a geospatial object are EPSG, PROJ.4, and WKT

Common CRSs

- EPSG:4326 - also known as WGS84, standard for GPS systems and Google Earth
- EPSG:3857 - a Mercator projection used in map tiles by Google Maps, Open Street Maps, etc.
- EPSG:27700 - also known as OSGB 1936, or the British National Grid, commonly used in Britain

```
st_crs(4326)$epsg

st_crs(3857)$epsg

st_crs(27700)$epsg
```

```
st_crs(4326)$Wkt

st_crs(3857)$Wkt

st_crs(27700)$Wkt
```

```
st_crs(4326)$proj4string

st_crs(3857)$proj4string

st_crs(27700)$proj4string
```

8.6 what about our dataset?

```
st_crs(CholeraDeaths)
```

```
cholera_4326 <- CholeraDeaths |>
  st_transform(4326)

st_bbox(cholera_4326)
```

```
ggplot(cholera_4326) +
  annotation_map_tile(type = "osm", zoomin = 0) +
  geom_sf(aes(size = Count), alpha = 0.7)
```

```
# add pump locations

pumps <- st_read(dsn, layer = "Pumps")

pumps_4326 <- pumps |>
  st_transform(4326)
```

```r
ggplot(cholera_4326) +
  annotation_map_tile(type = "osm", zoomin = 0) +
  geom_sf(aes(size = Count), alpha = 0.7) +
  geom_sf(data = pumps_4326, color = "red", size = 3)
```

Part IV

Code Performance

9 Code Performance

9.1 resources

- Advanced R - Chapters 22-25

9.2 code profiling

```
# load the package
library(profvis)  # for profiling R code and visualizing the results
```

```
# example 1

f <- function() {
  pause(0.1)
  g()
  h()
}

g <- function() {
  pause(0.1)
  h()
}

h <- function() {
  pause(0.1)
}

####

profvis(f())
```

```r
# example 2

# a slow, iterative random walk in R
random_walk_r <- function(n) {
  x <- numeric(n)
  x[1] <- 0
  for(i in 2:n) {
    x[i] <- x[i-1] + rnorm(1, mean = 0, sd = 1)
  }
  return(x)
}

####

profvis::profvis({
  # running a long simulation to give the profiler enough data
  sim_data <- random_walk_r(1e5)
})
```

9.3 improving performance with Rcpp

```r
# load the package
library(Rcpp)  # for integrating C++ code into R
```

```r
# define the C++ function as a string (or in a separate .cpp file)

Rcpp::cppFunction('
  NumericVector random_walk_cpp(int n) {
    NumericVector x(n);
    x[0] = 0;
    for(int i = 1; i < n; ++i) {
      x[i] = x[i-1] + R::rnorm(0, 1);
    }
    return x;
  }
')

# OR see below
```

```cpp
#include <Rcpp.h>
using namespace Rcpp;

// [[Rcpp::export]]

NumericVector random_walk_cpp_2(int n) {
  NumericVector x(n);
  x[0] = 0;
  for(int i = 1; i < n; ++i) {
    x[i] = x[i-1] + R::rnorm(0, 1);
  }
  return x;
}
```

```
profvis::profvis({
  # running a long simulation to give the profiler enough data
  sim_data_cpp <- random_walk_cpp(1e5)
})
```

```
# install.packages("bench")
library(bench)

# compare the R vs C++ implementations
results <- bench::mark(
  r_version = random_walk_r(10000),
  cpp_version = random_walk_cpp(10000),
  iterations = 100,
  check = FALSE # don't check for exact identical output due to random seeds
)
```

```
results |>
  dplyr::select(expression, min, median, `itr/sec`, mem_alloc)
```

```
plot(results)
```

9.4 parallelization with future and furrr

```
# load the packages
library(tictoc)   # for timing our code execution
```

```r
library(future)   # the core parallel backend architecture
library(furrr)    # a parallel implementation of the purrr family of functions
```

```r
future::availableCores()
```

```r
future::plan(multisession, workers = 2)
```

```r
simulate_median <- function(iteration, n = 1000) {
  # simulate heavily skewed data (e.g., Log-Normal)
  sample_data <- rlnorm(n, meanlog = 10, sdlog = 1)

  # return a tibble with the result
  tibble(
    iteration = iteration,
    median_val = median(sample_data)
  )
}
```

```r
# always test the function at least once before parallelizing

simulate_median(iteration = 1)
```

```r
tic("Sequential Purrr")
set.seed(405)
results_seq <- 1:1e4 |>
  purrr::map_dfr(.f = simulate_median)
toc()
```

```r
tic("Parallel Furrr")
set.seed(405)
results_par <- 1:1e4 |>
  furrr::future_map_dfr(.f = simulate_median, .options = furrr_options(seed = TRUE))
toc()
```

9.5 efficient file handling

```r
# load the packages
library(tidyverse)     # for data manipulation and writing CSV files
library(nanoparquet)   # for writing Parquet files in R
```

```r
write_csv(results_seq, "results_seq.csv")

write_parquet(results_seq, "results_seq.parquet")
```

```r
read_bench <- bench::mark(
  csv = read_csv("results_seq.csv", show_col_types = FALSE),
  parquet = read_parquet("results_seq.parquet"),
  iterations = 15,
  check = FALSE
)

read_bench |> select(expression, median)

plot(read_bench)
```

Part V

R Package

10 Building an R Package

10.1 check for available package names

```
# check available package names (chapter 4.1.2 of R Packages (2e))
install.packages(c("available", "pak"))
available("package_name")
pak::pkg_name_check("package_name")
```

10.2 create package

```
# start by creating a GitHub repo with the package name
# clone that repo to your Posit Cloud
```

```
# install some packages which are required for package development
install.packages(c("devtools", "checkmate", "tidyverse", "remotes"))
# devtools - meta package for package development in R
# checkmate - to build assertions (make sure that our functions are working fine)
# tidyverse - to build one of our functions
# remotes is a package that is required to install a package
```

```
library(devtools)
```

```
# create the package files (basic)
usethis::create_package("/cloud/project")
# make sure to hit the 'Yes' option and check the files/folders that have been created
# add your package name to the Package field in the DESCRIPTION file
# commit and push to GitHub
```

```
library(devtools)
```

10.3 create functions within the package

```
# existing function called strsplit that comes with base R
strsplit("alpha, bravo", split = ",")
# this returns by default a list which is annoying
strsplit("alpha, bravo", split = ",")[[1]]
# we wiLl create a new function called strsplit1 that returns a vector instead of a list
```

```
# make sure the diRectory is okay
getwd() # should see /cloud/project
```

```
# every function in an R package needs to be written in an .R script with the same name
# which will be stored in the R directory
usethis::use_r("strsplit1")
# type in function R script
strsplit1 <- function(x, split)
{
  strsplit(x, split = split)[[1]]
}
```

```
devtools::load_all()
# loads the package from source directly into the current environment
# temporarily builds and installs the package, allows us to verify our functions
```

```
# check if your functions works
strsplit1("alpha, bravo", split = ",")
# commit and push to GitHub
```

```
# create the min_max_scale function
usethis::use_r("min_max_scale")
# type in function R script
min_max_scale <- function(x)
{
  (x - min(x, na.rm = TRUE)) / (max(x, na.rm = TRUE) - min(x, na.rm = TRUE))
}

devtools::load_all()

# check
min_max_scale(1:5)
# commit and push to GitHub
```

```r
# create a personal theme (theme_academic)
usethis::use_r("theme_academic")
# type in function R script
theme_academic <- function(base_size = 12) {
  ggplot2::theme_classic(base_size = base_size) +
    ggplot2::theme(
      plot.title = ggplot2::element_text(face = "bold", hjust = 0.5),
      panel.grid.major = ggplot2::element_blank(),
      panel.grid.minor = ggplot2::element_blank()
    )
}
# specifically mention what external packages are used to build functions in your package
usethis::use_package("ggplot2")
# check Imports field in DESCRIPTION file

devtools::load_all()

# check
library(ggplot2)
p <- ggplot(mtcars) + geom_point(aes(x = mpg, y = hp)) + labs(title = "plot title")
p
p + theme_academic()
# commit and push to GitHub
```

```r
# so we have created the barebones version of all our three functions
```

```r
# we updated the DESCRIPTION file (changes to title, authors, description)
# most of the other things will get updated on their own
devtools::check()

# run the above check almost after every step
# should get 0 errors, 0 warnings, 0 notes
# right now we get a warning/note about license
```

10.4 add license

```r
# let's add a license
usethis:: use_mit_license()
# we won't change the LICENSE and LICENSE.md file by hand
```

```
devtools::check()  # check again
# commit and push to GitHub
```

10.5 add documentations

```
# we are now going add documentations to our functions
# go to the function .R script > Code> Insert roxygen skeleton
# make sure to have the @export tag
# think of a user when you are writing documentations
devtools::document()  # this needs to be run everytime we update the documentation
devtools::load_all()
devtools::check()
# commit and push to GitHub
```

10.6 add assertions

```
# we will work with assertions from the checkmate package
# first check of the function arguments so that it runs smoothly
# checkmate provides somewhat better messages
usethis::use_package("checkmate")
# we will write assertions for each input of each function
# see help pages of ?assert_
devtools::load_all()
# add assertions to each function and run devtools::load_all() after each change
# tryto break your function and see if the assertion works
devtools::document()
devtools::check()
# commit and push to GitHub
```

10.7 unit testing

```
# unit testing of functions
# think about every possible sceanrio where your function can break
usethis::use_testthat()
```

```
usethis::use_test("strsplit1")
# geneartes a new file inside the testthat directory inside tests directory
# we will delete the prepopulated message and write our own for our functions
# see help pages of ?expect_
# write your unit tests
devtools::load_all()
devtools::test()
```

```
usethis::use_test("min_max_scale")
devtools::load_all()
devtools::test()
```

```
usethis::use_test("theme_academic")
devtools::load_all()
devtools::test()
```

```
devtools::check()
# commit and push to GitHub
```

10.8 add dataset

```
# add dataset to our package that can be exported and used by our users
# first make sure that the dataset is created in your local Environment
toy_data <- data.frame(
  subject_id = 1:5,
  reaction_time = c(1.2, 0.9, 1.5, 0.8, 1.1),
  test_score = c(45, 82, 55, 91, 67),
  recalled_words = c(
    "cat, dog, mouse",
    "apple, banana",
    "car, truck, bike, train",
    "red, blue, green",
    "sun, moon"
  )
)
```

```
usethis::use_data(toy_data)
# if exporting big daatsets, look into the compress argument of use_this
```

```
# we will save the data generating script into a file (for reproducibility)
# very important if you generate datasets that need random generation (set a seed)
usethis::use_data_raw("toy_data_code")
# anytime you update the toy_data with new code, make sure to update the toy_data_code file w
```

```
# documentation for toy_data
usethis::use_r("toy_data")
# seems to be done manually
# this will build the documentation
# make sure that the last line has the name of the dataset inside ""
# mention the units of your variables
```

```
# if we export multiple datasets, R puts them all in sysdata.rda file
# in the NAMESPACE, it doesn't specifically mention the dataset to be exported

devtools::document()
devtools::load_all()  # display the dataset and check help page
devtools::check()
# commit and push to GitHub
```

```
# for any file that is typically not an rda file, like csv files
# we will need to create a directory called inst/extdata
```

10.9 add README

```
# we will build the README for the GitHub site and also for our website
usethis::use_readme_rmd()
# try not to install any other pacakges (since those packages have not been mentioned in Impo
devtools::build_readme()
# use this build function repreatedly whenever you update the README.Rmd
devtools::check()
# commit and push to GitHub
# check GitHub
```

10.10 add/build vignettes

```
# build a vignette
# these are specific instrcutions/notes about very specific part of your package
# for example check out the ggplot2 vignettes from Articles dropdown of the package website
usethis::use_vignette(name = "intro-to-toypackage", title = "Introduction to toypackage")
devtools::build_vignettes()  # to convert the RMd into website readable html
devtools::check()
# commit and push to GitHub
```

10.11 build package website

```
# build the package website
# pkgdown package this is how the website will be built
usethis::use_pkgdown()  # run this only once
# the website will also use GitHub pages like it did for our personal website
pkgdown::build_site()   # run this everytime there is an update to anything inside your packa
usethis::use_pkgdown_github_pages()  # notice the changes, run this only once
devtools::check()
# commit and push to GitHub
```